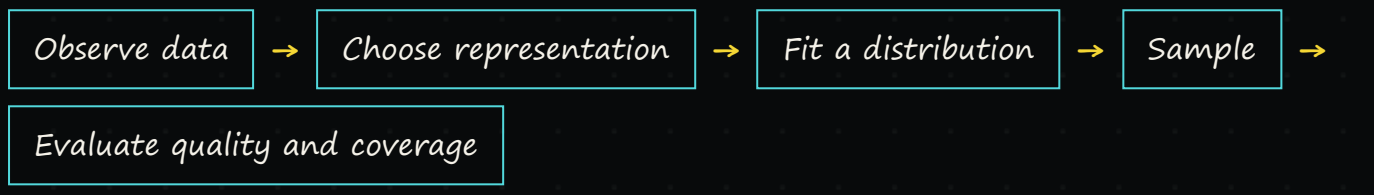
Generative Modeling

Learn a useful approximation to a data-generating distribution, then sample new observations. Generation needs both realistic individual samples and coverage of the distribution, not memorization alone.

Original study notes based on David Foster, Second edition, 2023. Book pp. 3–22. Source APIs, services, experiments, and model comparisons reflect the book's publication period. These are explanatory summaries, not verbatim reproductions or a claim that every source detail is retained.



Detailed Notes

Generative versus discriminative modeling

Source: book pp. 3–8 / PDF p. 31

A discriminative model predicts a label or decision from an input, commonly $p(y|x)$. A generative model describes a distribution over observations, $p(x)$, or a joint/conditional distribution from which observations can be sampled. A conditional generator can produce x given y, such as an image of a requested class.

Generation is not restricted to images: text, music, trajectories, and other structured objects can be modeled. A convincing example is insufficient evidence of learning; test realism, diversity, controllability, and whether training items are being copied. The rise of deep learning made flexible representations and large-scale generators practical, but generation and intelligence are not identical concepts.

The first generative model and its assumptions

Source: book pp. 9–11 / PDF p. 37

The introductory example estimates a distribution from simple observations rather than immediately using a large neural network. Choose a sample space, represent the observed records, estimate probabilities, and draw new samples. The fitted distribution can only express structures allowed by the model family.

Separate the unknown population distribution from its finite training sample. Overfitting puts too much probability on the observed examples; underfitting misses real patterns. Held-out data and inspection of generated outputs expose different problems. A model that assigns plausible densities but samples badly, or generates attractive samples while missing most of the data, needs further investigation.

Representation learning and latent spaces

Source: book pp. 12–14 / PDF p. 40

Raw pixels or tokens are often a poor space for reasoning about meaningful changes. Representation learning maps observations into features or latent variables that make variation easier to model. A decoder/generator can map a latent point back to the observation space.

A useful latent representation can support interpolation, editing, and reuse across tasks. Low dimension alone does not guarantee semantic organization: nearby coordinates may not produce meaningful observations unless training imposes appropriate structure. Distinguish an inferred code for an observed example from a latent variable sampled from a chosen prior.

Core probability theory and maximum likelihood

Source: book pp. 15–17 / PDF p. 43

A probability mass function applies to discrete outcomes; a probability density applies to continuous values, whose probability is obtained by integration. A density can exceed one without violating probability rules. Joint, marginal, and conditional distributions relate through the product rule and marginalization.

> $p(x,y) = p(x|y)p(y)$
> $p(x|y) = p(y|x)p(x)/p(y)$
> Maximum likelihood: maximize $\Sigma_i \log p_\theta(x_i)$

Log likelihood turns products into sums and improves numerical stability. Minimizing negative log likelihood is not the same as minimizing a perceptual distance between one generated image and an arbitrary training image. The objective must match the probabilistic model.

Taxonomy of generative models

Source: book pp. 18–19 / PDF p. 46

VAEs use latent variables and an approximate inference model. **GANs** learn through an adversarial discriminator/critic without an easily evaluated likelihood. **Autoregressive models** factor a joint distribution into ordered conditionals. **Flows** transform a simple density through invertible mappings. **Energy models** learn unnormalized scores. **Diffusion models** learn to reverse progressive corruption.

Compare tractability of likelihood, sampling speed, architectural restrictions, sample quality, and training stability. These are families of trade-offs, not a leaderboard where one method wins every task. Later chapters show how families can be combined.

Codebase, Docker, GPU, and reproducibility

Source: book pp. 20–22 / PDF p. 48

The book provides a practical TensorFlow/Keras codebase. Clone the matching edition, inspect dependency versions, and use an isolated environment or the documented container. A container packages software dependencies; GPU execution additionally requires compatible host drivers and a supported runtime.

Record dataset versions, preprocessing, seeds, architecture, optimizer, checkpoints, and hardware. Confirm data loading, one training step, saving, reloading, and sampling on a small run before a long experiment. Historical package/API examples may need updating, but changes should preserve the stated learning objective and tensor shapes.

Revision Check

1. How does learning $p(x)$ differ from learning $p(y|x)$?
2. Why is a probability density allowed to exceed one?
3. How would you distinguish memorization from useful sampling?
4. Which model families expose an exact tractable likelihood?

A generative model is a distributional hypothesis. Evaluate what it represents, what it omits, and how its samples are produced.

Source links refer to the supplied PDF. Every contents-page topic is mapped in coverage.md. Consult the book for complete implementations, figures, exercises, citations, and experiment evidence.